

Contents

Experiment 1: Design the Smart City Model Using IoT Devices..... 1

Experiment2: Ascending Order Using Embedded C 3

Experiment 3: Demonstrate External LED Blinking Using Arduino Programming 4

Experiment 4: Demonstrate the Installation of 8051 Assembly Language and the Keil Simulator with a Simple Program .. 4

Experiment 5: Write a Program to Perform Descending Order Using Embedded C..... 5

Experiment 6: Patient Data Transfer System Between Patient and Hospital Using IoT 6

Experiment 7: Exploring Different Communication (Bluetooth) in IoT Using Wokwi..... 7

Experiment 8: Write a Program to Perform Comparison Operations in 8051 Using a Simulator 8

Experiment 9: Design and Implement a Simple Home Automation System Using Embedded Devices 9

Experiment 10: Design and Implement an Automatic Irrigation System Using IoT 10

Experiment 11: Design and Implement the System to Control Temperature in the Room Using IoT 10

Experiment 13: Write the Concept to Set the Cloud Platform to Log the Data 11

Experiment 14: Case Study on Raspberry Pi Boards and Their Applications..... 12

Experiment 15: Case Study on Arduino Boards and Their Applications 14

Experiment 18: Write a Program for Analog Read Operation in Arduino 16

Experiment 19: Implement the Interfacing of a Gas Sensor with Raspberry Pi Pico and Demonstrate Real-Time Gas Level Monitoring 17

Experiment 20: Interface a DHT22 Temperature and Humidity Sensor with a Raspberry Pi and Demonstrate Real-Time Temperature and Humidity Data Monitoring..... 18

Experiment 1: Design the Smart City Model Using IoT Devices

Aim

To design and develop a smart city model using IoT devices for monitoring and controlling city services such as street lights, traffic systems, environmental monitoring, and waste management.

Components Required

- **Arduino Uno / ESP32**
- **Wi-Fi Module (ESP8266 if needed)**
- **LED lights**
- **LDR sensor**
- **Ultrasonic sensor**
- **DHT11/DHT22 sensor**
- **Breadboard**
- **Jumper wires**
- **Buzzer**
- **Power supply**

Theory

A Smart City uses IoT technology to improve the quality of urban services. Sensors collect real-time data from different areas of the city and send the information to cloud platforms or monitoring systems. IoT helps in smart traffic control, automatic street lighting, pollution monitoring, waste management, and energy saving.

Procedure

Step 1: Circuit Setup

1. Connect LEDs to Arduino pins for smart street light demonstration.
 2. Connect the LDR sensor to detect light intensity.
 3. Connect the DHT sensor for temperature monitoring.
 4. Connect the ultrasonic sensor for smart parking/waste monitoring.
 5. Provide power supply to the Arduino board.
-

Step 2: Programming

1. Open Arduino IDE.
 2. Write the program for reading sensor values.
 3. Program the LEDs to turn ON/OFF automatically based on sensor input.
 4. Upload the code to Arduino.
-

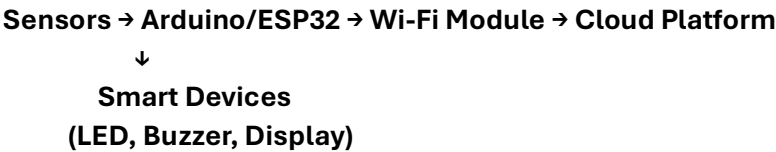
Step 3: IoT Communication

1. Connect the ESP32/ESP8266 to Wi-Fi.
 2. Send sensor data to a cloud platform such as ThingSpeak or Blynk.
 3. Monitor the data in real time.
-

Step 4: Testing

1. Test automatic street light operation using the LDR sensor.
 2. Check temperature values from the DHT sensor.
 3. Verify obstacle/distance detection using the ultrasonic sensor.
 4. Observe data updates on the cloud platform.
-

Block Diagram



Advantages

- Reduces power consumption
 - Provides real-time monitoring
 - Improves city management
 - Supports automation
 - Reduces human effort
-

Applications

- Smart traffic system
 - Smart parking
 - Smart street lighting
 - Pollution monitoring
 - Smart waste management
-

Result

Thus, the smart city model using IoT devices was successfully designed and implemented for monitoring and automation purposes.

Experiment2: Ascending Order Using Embedded C

Aim

To write and execute an Embedded C program to arrange numbers in ascending order.

Requirements

- Arduino Uno
 - USB Cable
 - Arduino IDE
 - Computer/Laptop
-

Procedure

1. Open Arduino IDE.
 2. Connect the Arduino board to the computer.
 3. Create a new sketch.
 4. Write the Embedded C program for ascending order.
 5. Verify the program.
 6. Upload the program to Arduino.
 7. Open the Serial Monitor.
 8. Observe the sorted numbers in ascending order.
-

Program

```
void setup() {  
    Serial.begin(9600);  
  
    int a[5]={5,4,3,2,1};  
    int i,j,t;  
  
    for(i=0;i<5;i++)  
        for(j=i+1;j<5;j++)  
            if(a[i]>a[j]) {  
                t=a[i];  
                a[i]=a[j];  
                a[j]=t;  
            }  
  
    for(i=0;i<5;i++)  
        Serial.println(a[i]);  
}  
  
void loop() {}
```

Output

```
1  
2  
3  
4  
5
```

Result

Thus, the Embedded C program for arranging numbers in ascending order was executed successfully

Experiment 3: Demonstrate External LED Blinking Using Arduino Programming

Aim

To demonstrate external LED blinking using Arduino programming in Tinkercad simulation.

Software Required

- Tinkercad Circuits
 - Arduino Uno
 - LED
 - Resistor
 - Jumper wires
-

Procedure

1. Open [Tinkercad Circuits](#)
 2. Create a new circuit.
 3. Add Arduino Uno, LED, and resistor.
 4. Connect the LED to pin 13 and GND.
 5. Open the code editor.
 6. Write the Arduino program.
 7. Start the simulation.
 8. Observe the LED blinking continuously.
-

Program

```
void setup() {  
  pinMode(13, OUTPUT);  
}  
  
void loop() {  
  digitalWrite(13, HIGH);  
  delay(1000);  
  
  digitalWrite(13, LOW);  
  delay(1000);  
}
```

Output

The LED blinks continuously in the Tinkercad simulation.

Result

Thus, external LED blinking using Arduino programming was demonstrated successfully using Tinkercad simulation.

Experiment 4: Demonstrate the Installation of 8051 Assembly Language and the Keil Simulator with a Simple Program

Aim

To demonstrate the installation of 8051 assembly language and the Keil simulator with a simple program.

Software Required

- Keil uVision Software
- Computer/Laptop

Procedure

1. Download and install Keil uVision software.
2. Open Keil software.
3. Create a new project.
4. Select the 8051 microcontroller.
5. Create a new assembly file.
6. Write the simple assembly language program.
7. Save the file with .asm extension.
8. Build the program.
9. Run the simulation and observe the output.

Program

```
MOV A,#05H
MOV R0,#03H
ADD A,R0
END
```

Output

Accumulator Value = 08H

Result

Thus, the installation of 8051 assembly language and the Keil simulator was demonstrated successfully with a simple program.

Experiment 5: Write a Program to Perform Descending Order Using Embedded C

Aim

To write and execute an Embedded C program to arrange numbers in descending order.

Software Required

- Arduino IDE
- Tinkercad Simulation
- Arduino Uno

Procedure

1. Open Tinkercad Circuits.
2. Create a new circuit with Arduino Uno.
3. Open the code editor.
4. Write the Embedded C program for descending order.
5. Start the simulation.
6. Open the Serial Monitor.
7. Observe the numbers arranged in descending order.

Program

```
void setup() {
  Serial.begin(9600);

  int a[5]={1,2,3,4,5};
  int i,j,t;
```

```
for(i=0;i<5;i++)
  for(j=i+1;j<5;j++)
    if(a[i]<a[j]) {
      t=a[i];
      a[i]=a[j];
      a[j]=t;
    }

for(i=0;i<5;i++)
  Serial.println(a[i]);
}

void loop() {}
```

Output
5
4
3
2
1

Result
Thus, the Embedded C program for descending order was executed successfully using Tinkercad simulation.

Experiment 6: Patient Data Transfer System Between Patient and Hospital Using IoT

Introduction
IoT-based healthcare systems help hospitals monitor patient health conditions remotely. Sensors attached to the patient collect health data such as temperature, heartbeat, and oxygen level. The collected data is transferred to hospitals through the internet for continuous monitoring and emergency response.

- Objective**
The main objective of this system is:
- To monitor patient health remotely
 - To transfer patient data to hospitals in real time
 - To reduce hospital visits
 - To provide quick medical response during emergencies
-

- Components Used**
- Arduino Uno / ESP32
 - Temperature Sensor
 - Pulse Sensor
 - Wi-Fi Module
 - Buzzer
 - Cloud Platform (ThingSpeak/Blynk)
 - Internet Connection
-

Working Principle
The sensors continuously monitor patient health conditions. The Arduino or ESP32 reads the sensor values and sends the data to the hospital server using Wi-Fi modules.
Examples:

- Temperature sensor measures body temperature.
- Pulse sensor measures heart rate.
- Cloud platform stores patient data.
- Doctors can monitor patient health remotely.
- Alerts are generated during abnormal conditions.

Block Diagram

Patient Sensors → Arduino/ESP32 → Wi-Fi Module → Hospital Server/Cloud

↓

Alert System

Features

- Real-time patient monitoring
- Wireless data transfer
- Emergency alert system
- Cloud-based health monitoring
- Remote healthcare support

Advantages

- Reduces hospital workload
- Saves time
- Provides quick emergency response
- Supports remote patient monitoring
- Improves healthcare services

Applications

- Smart hospitals
- Remote healthcare systems
- Elderly patient monitoring
- Emergency medical services
- Home healthcare systems

Conclusion

Thus, the IoT-based patient data transfer system helps hospitals monitor patients remotely and improves healthcare management through real-time data communication and emergency support.

Experiment 7: Exploring Different Communication (Bluetooth) in IoT Using Wokwi

Aim

To explore Bluetooth communication in IoT using Wokwi simulation.

Requirements

- ESP32 (Wokwi Simulator)
- LED
- 220Ω Resistor
- Wokwi Simulation Platform

Procedure

1. Open Wokwi and create a new ESP32 project.
2. Connect the LED to GPIO 2 and GND through a resistor.
3. Write the program in the Wokwi code editor.

4. Click Start Simulation.
 5. Open the Serial Monitor.
 6. Type the following commands:
 - 1 → LED ON
 - 0 → LED OFF
 7. Observe the LED operation based on the entered value.
-

Source Code

```
int ledPin = 2;
char data;

void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);
  Serial.println("Enter 1 for ON, 0 for OFF");
}

void loop() {
  if (Serial.available()) {
    data = Serial.read();

    if (data == '1') {
      digitalWrite(ledPin, HIGH);
      Serial.println("LED ON");
    }
    else if (data == '0') {
      digitalWrite(ledPin, LOW);
      Serial.println("LED OFF");
    }
  }
}
```

Output

Enter 1 for ON, 0 for OFF

1 → LED ON
0 → LED OFF

Result

Thus, the simulation for Bluetooth communication in IoT using Wokwi was performed successfully.

Experiment 8: Write a Program to Perform Comparison Operations in 8051 Using a Simulator

Aim

To write and execute a program for comparison operations in 8051 using the Keil simulator.

Software Required

- Keil uVision Software
 - 8051 Simulator
-

Procedure

1. Open Keil uVision software.
2. Create a new 8051 project.
3. Select the 8051 microcontroller.
4. Create a new assembly file.
5. Write the comparison program.
6. Save the file with .asm extension.
7. Build the program.
8. Run the simulation and observe the result.

Program

```
MOV A,#05H  
MOV R0,#03H
```

```
CJNE A,R0,NOTEQUAL
```

```
NOTEQUAL:  
END
```

Output

```
05H is not equal to 03H
```

Result

Thus, the comparison operation in 8051 using the simulator was executed successfully.

Experiment 9: Design and Implement a Simple Home Automation System Using Embedded Devices

Aim

To control home appliances using ESP32 and Blynk application.

Requirements

- ESP32
- LED
- 220Ω Resistor
- Blynk App
- WiFi Connection

Procedure

Step 1: Circuit Connection

1. Open Wokwi/Tinkercad or use ESP32 hardware.
2. Connect LED to GPIO pin of ESP32.
3. Provide power supply to the circuit.

Step 2: Blynk Setup

1. Install Blynk IoT application.
2. Create a new template.
3. Select ESP32 and WiFi connection.
4. Copy the Authentication Token.

Step 3: Configure Datastream

1. Create Virtual Pin V0.
2. Add button widget in dashboard.
3. Save and apply settings.

Step 4: Arduino Programming

1. Open Arduino IDE.

2. Install ESP32 board package and Blynk library.
3. Write and upload the program.

Step 5: Execution

1. Start simulation/run hardware.
2. Control LED using Blynk app.
3. Observe ON/OFF operation.

Result

Thus, the simple home automation system using ESP32 and Blynk was implemented successfully.

Experiment 10: Design and Implement an Automatic Irrigation System Using IoT

Aim

To design and implement an automatic irrigation system using IoT.

Requirements

- ESP32
- Soil Moisture Sensor
- Water Pump/LED
- Blynk App
- WiFi Connection

Procedure

Step 1: Circuit Connection

1. Open Wokwi/Tinkercad or use ESP32 hardware.
2. Connect soil moisture sensor to ESP32.
3. Connect motor/LED to GPIO pin.

Step 2: Blynk Setup

1. Install Blynk IoT application.
2. Create a new template.
3. Copy the Authentication Token.

Step 3: Configure Datastream

1. Create Virtual Pin V0.
2. Add monitoring widget in dashboard.

Step 4: Arduino Programming

1. Open Arduino IDE.
2. Install required libraries.
3. Write and upload the irrigation program.

Step 5: Execution

1. Start the simulation.
2. Monitor soil moisture values.
3. Observe automatic motor ON/OFF operation.

Result

Thus, the automatic irrigation system using IoT was implemented successfully.

Experiment 11: Design and Implement the System to Control Temperature in the Room Using IoT

Aim

To design and implement a temperature control system using IoT.

Requirements

- ESP32
- DHT11/DHT22 Sensor
- Fan/LED
- Blynk App
- WiFi Connection

Procedure

Step 1: Circuit Connection

1. Open Wokwi/Tinkercad or use ESP32 hardware.
2. Connect DHT sensor to ESP32.
3. Connect fan/LED to GPIO pin.

Step 2: Blynk Setup

1. Install Blynk IoT application.
2. Create a new template.
3. Copy the Authentication Token.

Step 3: Configure Datastream

1. Create Virtual Pin V0.
2. Add monitoring widget in dashboard.

Step 4: Arduino Programming

1. Open Arduino IDE.
2. Install DHT and Blynk libraries.
3. Write and upload the program.

Step 5: Execution

1. Start the simulation.
2. Monitor temperature values.
3. Observe automatic fan/LED control.

Result

Thus, the temperature control system using IoT was implemented successfully.

Experiment 13: Write the Concept to Set the Cloud Platform to Log the Data

Aim

To set up a cloud platform for logging sensor data using IoT devices.

Requirements

- ESP32 / Arduino
 - WiFi Connection
 - Sensor Module
 - ThingSpeak/Blynk Cloud
 - Arduino IDE
-

Procedure

Step 1: Cloud Setup

1. Create account in ThingSpeak/Blynk.
2. Create a new channel/template.
3. Generate API key/Auth token.

Step 2: Circuit Connection

1. Connect sensor to ESP32.
2. Connect ESP32 to computer.

Step 3: Arduino Programming

1. Open Arduino IDE.
2. Install required libraries.
3. Enter WiFi credentials and API key.

4. Upload the program.

Step 4: Execution

1. Connect ESP32 to WiFi.
2. Send sensor data to cloud platform.
3. Observe data in graph/dashboard.

Result

Thus, the cloud platform for logging IoT sensor data was configured successfully.

Experiment 14: Case Study on Raspberry Pi Boards and Their Applications

Aim

To study Raspberry Pi boards and their applications in embedded systems and IoT.

Introduction

Raspberry Pi is a low-cost, credit-card-sized single-board computer developed by the Raspberry Pi Foundation in the United Kingdom. It was mainly designed to promote computer science education and programming skills among students. Due to its compact size, low power consumption, and powerful processing capability, Raspberry Pi is widely used in embedded systems, robotics, automation, artificial intelligence, and Internet of Things (IoT) applications.

Raspberry Pi can run operating systems such as Raspberry Pi OS (Linux-based), Ubuntu, and Windows IoT Core. It supports programming languages like Python, C, C++, and Java. The board contains GPIO (General Purpose Input Output) pins that help interface sensors, motors, relays, and other electronic devices.

Today, Raspberry Pi is used in industries, educational institutions, research laboratories, and smart automation systems because of its flexibility and easy hardware interfacing.

Features of Raspberry Pi

- Small and compact design
 - Low power consumption
 - Built-in WiFi and Bluetooth support
 - HDMI display support
 - USB ports for keyboard and mouse
 - GPIO pins for sensor interfacing
 - Supports Linux operating system
 - Supports Python and Embedded C programming
 - Camera and display interface support
 - Suitable for IoT and AI applications
-

Types of Raspberry Pi Boards

Raspberry Pi 3

- Quad-core processor
- 1GB RAM
- Built-in WiFi and Bluetooth
- Used for IoT and automation projects

Raspberry Pi 4

- Improved processor performance
- Available with 2GB, 4GB, and 8GB RAM
- Supports dual display
- USB 3.0 support
- Suitable for advanced AI and multimedia applications

Raspberry Pi Zero

- Smallest Raspberry Pi board
- Low power consumption
- Compact and portable
- Used in wearable and compact IoT devices

Raspberry Pi Pico

- Microcontroller-based board
- Uses RP2040 chip
- Supports MicroPython and Embedded C
- Used in sensor interfacing and embedded applications

Components of Raspberry Pi

Processor

The processor performs all computation and data processing operations.

RAM

RAM stores temporary data and helps execute programs efficiently.

GPIO Pins

GPIO pins are used for interfacing LEDs, sensors, relays, motors, and other external devices.

USB Ports

Used to connect keyboard, mouse, storage devices, and other peripherals.

HDMI Port

Used to connect monitors and display devices.

WiFi and Bluetooth

Provides wireless communication for IoT applications.

SD Card Slot

Stores the operating system and files required for Raspberry Pi operation.

Working Principle

Raspberry Pi works as a mini-computer that can read data from sensors, process the data, and control output devices. Sensors connected through GPIO pins send signals to the Raspberry Pi. The processor analyzes the input and performs actions such as turning ON LEDs, controlling motors, sending data to cloud platforms, or displaying information on a monitor.

For IoT applications, Raspberry Pi connects to the internet through WiFi or Ethernet and transfers data to cloud platforms like ThingSpeak, Firebase, or Blynk.

Applications of Raspberry Pi

Home Automation

Used to control lights, fans, and home appliances remotely through mobile applications.

Smart Surveillance System

Used with cameras for face recognition, motion detection, and security monitoring.

Robotics

Controls robotic movements and sensor operations.

IoT Projects

Used for cloud communication and real-time sensor monitoring.

Weather Monitoring System

Measures temperature, humidity, and environmental conditions.

Smart Healthcare System

Monitors patient health data such as heartbeat and temperature.

Industrial Automation

Used for machine monitoring and industrial control systems.

Artificial Intelligence and Machine Learning

Used for image processing, object detection, and AI-based applications.

Advantages of Raspberry Pi

- Low cost and affordable

- Easy hardware interfacing
 - Compact and portable
 - Supports multiple programming languages
 - Suitable for educational purposes
 - Supports IoT and AI applications
 - Large community support
 - Energy efficient
-

Limitations of Raspberry Pi

- Lower performance compared to full desktop computers
 - Limited processing capability for high-end applications
 - Requires external storage through SD card
 - GPIO pins support limited current
-

Future Scope

Raspberry Pi is becoming increasingly important in modern technologies such as:

- Artificial Intelligence
- Internet of Things
- Smart Cities
- Industrial Automation
- Robotics
- Healthcare Systems
- Edge Computing

Future developments may include more powerful processors, improved AI support, and enhanced wireless communication.

Experiment 15: Case Study on Arduino Boards and Their Applications

Aim

To study Arduino boards and their applications in embedded systems and IoT.

Introduction

Arduino is an open-source microcontroller platform used for embedded systems, automation, robotics, and Internet of Things (IoT) applications. Arduino boards are easy to program and interface with sensors, motors, LEDs, displays, and communication modules.

Arduino uses Embedded C/C++ programming through the Arduino IDE. Due to its simple hardware and software environment, Arduino is widely used by students, beginners, engineers, and researchers for developing electronic projects and automation systems.

Arduino boards can receive input from sensors, process the data, and control output devices based on the program logic.

Features of Arduino

- Open-source platform
 - Easy programming using Arduino IDE
 - Supports Embedded C/C++
 - Supports sensor interfacing
 - USB programming support
 - Digital and analog I/O pins
 - Low power consumption
 - Suitable for IoT and automation projects
-

Types of Arduino Boards

Arduino Uno

Arduino Uno is the most commonly used Arduino board for beginners and embedded projects.

Specifications

- **Microcontroller: ATmega328P**
 - **Operating Voltage: 5V**
 - **Digital I/O Pins: 14**
 - **Analog Input Pins: 6**
 - **Clock Speed: 16 MHz**
-

Arduino Nano

Arduino Nano is a compact version of Arduino Uno mainly used in small embedded applications.

Specifications

- **Microcontroller: ATmega328P**
 - **Operating Voltage: 5V**
 - **Compact size**
 - **Breadboard friendly**
 - **USB support available**
-

Arduino Mega

Arduino Mega is used for large projects requiring more pins and memory.

Specifications

- **Microcontroller: ATmega2560**
 - **Digital I/O Pins: 54**
 - **Analog Input Pins: 16**
 - **Large memory capacity**
 - **Clock Speed: 16 MHz**
-

Arduino Leonardo

Arduino Leonardo contains built-in USB communication support.

Specifications

- **Microcontroller: ATmega32U4**
 - **Built-in USB communication**
 - **20 digital I/O pins**
 - **12 analog input pins**
-

Arduino LilyPad

Arduino LilyPad is specially designed for wearable electronics and smart clothing applications.

Specifications

- **Lightweight and flexible design**
 - **Sewable pads for stitching**
 - **Low power consumption**
 - **Supports wearable sensors**
-

Components of Arduino Uno

Microcontroller

The ATmega328P microcontroller performs all processing operations.

Digital Pins

Used for digital input and output operations.

Analog Pins

Used to read analog sensor values.

USB Port

Used for programming and power supply.

Power Jack

Provides external power supply to the board.

Reset Button

Used to restart the Arduino board.

Working Principle

Arduino boards read data from sensors and process the information using the microcontroller. Based on the programmed instructions, Arduino controls output devices such as LEDs, motors, relays, buzzers, and displays. For IoT applications, Arduino can communicate with cloud platforms using WiFi or Bluetooth modules.

Applications of Arduino

- Home automation systems
 - Robotics and automation
 - Internet of Things (IoT) projects
 - Smart irrigation systems
 - Healthcare monitoring systems
 - Industrial automation
 - Smart city applications
 - Environmental monitoring systems
 - Security and surveillance systems
 - Wearable electronic devices
 - Educational and research projects
 - Sensor interfacing applications
 - Wireless communication systems
 - Smart energy management systems
 - Artificial intelligence and machine learning projects
-

Advantages of Arduino

- Easy to learn and program
 - Low cost
 - Large community support
 - Supports many sensors and modules
 - Compact and portable
 - Ideal for beginners and IoT projects
-

Limitations of Arduino

- Limited processing power
 - Limited memory compared to Raspberry Pi
 - Not suitable for high-end computing applications
-

Future Scope

Arduino is widely used in:

- Smart cities
- Robotics
- Artificial Intelligence
- IoT systems
- Automation industries
- Wearable electronics

Future Arduino boards may include improved wireless communication and AI support.

Experiment 18: Write a Program for Analog Read Operation in Arduino

Aim

To perform analog read operation using a gas sensor in Arduino using Tinkercad simulation.

Requirements

- Arduino Uno
 - Gas Sensor (MQ2)
 - Jumper Wires
 - Tinkercad Simulation
-

Procedure

1. Open [Tinkercad Circuits](#)
 2. Create a new circuit.
 3. Add Arduino Uno and MQ2 gas sensor.
 4. Connect the analog output pin of MQ2 sensor to A0 of Arduino.
 5. Connect VCC and GND properly.
 6. Open the code editor.
 7. Write the Arduino program.
 8. Click Start Simulation.
 9. Open Serial Monitor and observe gas sensor values.
-

Program

```
void setup() {  
  Serial.begin(9600);  
}  
  
void loop() {  
  
  int gasValue = analogRead(A0);  
  
  Serial.println(gasValue);  
  
  delay(500);  
}
```

Output

Gas Sensor Value:
0 to 1023

Result

Thus, the analog read operation using gas sensor in Arduino was simulated successfully in Tinkercad.

Experiment 19: Implement the Interfacing of a Gas Sensor with Raspberry Pi Pico and Demonstrate Real-Time Gas Level Monitoring

Aim

To interface a gas sensor with Raspberry Pi Pico and demonstrate real-time gas level monitoring.

Requirements

- Raspberry Pi Pico
 - MQ2 Gas Sensor
 - Jumper Wires
 - Breadboard
 - Thonny IDE / Wokwi Simulation
-

Procedure

1. Open Wokwi or connect Raspberry Pi Pico hardware.

2. Connect MQ2 gas sensor analog output to ADC pin of Pico.
 3. Connect VCC and GND properly.
 4. Open Thonny IDE or Wokwi code editor.
 5. Write the MicroPython program.
 6. Run the program.
 7. Observe gas sensor values in serial output.
-

Program

```
from machine import ADC, Pin
from time import sleep
```

```
gas = ADC(26)
```

```
while True:
```

```
    value = gas.read_u16()
    print("Gas Value:", value)
    sleep(1)
```

Output

Gas Value: 12000

Gas Value: 18000

Gas Value: 25000

Result

Thus, the interfacing of gas sensor with Raspberry Pi Pico for real-time gas level monitoring was implemented successfully.

Experiment 20: Interface a DHT22 Temperature and Humidity Sensor with a Raspberry Pi and Demonstrate Real-Time Temperature and Humidity Data Monitoring

Aim

To interface a DHT22 temperature and humidity sensor with a Raspberry Pi and demonstrate real-time temperature and humidity monitoring.

Requirements

- Raspberry Pi
 - DHT22 Sensor
 - Breadboard
 - Jumper Wires
 - Python / Thonny IDE
-

Procedure

1. Connect the DHT22 sensor to the Raspberry Pi GPIO pins.
 2. Connect VCC to 3.3V and GND to ground.
 3. Connect the data pin of DHT22 to a GPIO pin.
 4. Open Thonny IDE on Raspberry Pi.
 5. Write the Python program.
 6. Run the program.
 7. Observe the temperature and humidity values in real time.
-

Program

```
import Adafruit_DHT
import time

sensor = Adafruit_DHT.DHT22
pin = 4

while True:
    humidity, temperature = Adafruit_DHT.read_retry(sensor, pin)

    if humidity is not None and temperature is not None:
        print("Temperature = %.1f C" % temperature)
        print("Humidity = %.1f %% " % humidity)
    else:
        print("Sensor reading failed")

    time.sleep(2)
```

Output

```
Temperature = 28.5 C
Humidity = 62.3 %
Temperature = 28.7 C
Humidity = 62.1 %
```

Result

Thus, the DHT22 temperature and humidity sensor was interfaced with Raspberry Pi and real-time data monitoring was demonstrated successfully